CodeCraft Academy
Empowering Futures with Every Keystroke

C Programming

Presented by Mannat Doultani

CodeCraft Academy
Empowering Futures with Every Keystroke

Overview

- Introduction
  - History & Development
  - Features of C
  - Application
  - Relevance

- First C Program
- Understanding printf()
- Compiling & Running Prg.
- Structure of C Program
- Algorithm & Flowchart

CodeCraft Academy
Empowering Futures with Every Keystroke

1. Introduction to C Programming

```
#include <stdio.h>
int main() {
    printf("Helio, C!");
    return 0;
}
```

1 What is C Programming?

- C is a general-purpose, procedural programming language.
- It is used to write software programs that can run on a variety of platforms.
- C provides a powerful set of features with a relatively simple and compact syntax.
- It is considered a foundational language for learning programming.

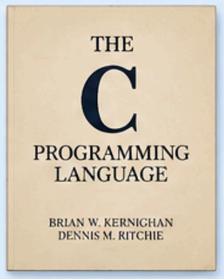


2 History and development of C

1970	Developed by Dennis Ritchie at Bell Labs (AT&T), USA.
1972	C emerged from an earlier language called B.
1978	The book *The C Programming Language* by Brian Kernighan and Dennis Ritchie popularised C.
1980s	C became widely used for system software development.
1989	The ANSI C standard (C89) was published.
1999	C99 introduced new features.
2011	C11 introduced further improvements.



Dennis Ritchie
Creator of C
(1941 – 2011)



3 Features of C

- **Simple and structured**
  Easy to learn and use.
- **Efficient and fast**
  Provides high performance.
- **Low-level access**
  Allows direct memory and hardware manipulation.
- **Portable**
  Programs can run on different platforms with minimal changes.
- **Rich set of operators**
  Supports a wide range of operations.
- **Large standard library**
  Provides many built-in functions.
- **Modular programming**
  Supports functions and separate source files.

4 Applications of C

- **Operating Systems**
  e.g., UNIX, Linux, Windows (parts)
- **System Software**
  Compilers, interpreters, device drivers
- **Embedded Systems**
  e.g., IoT devices, automotive systems, appliances
- **Database Systems**
  e.g., MySQL (core components)
- **Networking Software**
  Protocols, servers, tools
- **Game Development**
  Game engines and graphics programming
- **Scientific and Engineering Applications**
  Simulations, numerical computing

5 Why C is still relevant?

- Form the foundation for many modern programming languages (e.g., C++, Java, Python).
- Widely used in system software, embedded systems and performance-critical applications.
- Provides better understanding of computer architecture and memory management.
- Offers high efficiency and low-level control.
- Still used in modern software development, operating systems, compilers and many real-world applications.
- Helps develop strong problem-solving and programming skills.

CodeCraft Academy
Empowering Futures with Every Keystroke

2. Your First C Program

1 Writing a Hello World program

The Hello World program is the first program in any programming language. It simply displays the text “Hello, World!” on the screen.

hello_world.c

```c
#include <stdio.h>

int main()
{
    printf("Hello, World!\n");
    return 0;
}
```

2 Understanding printf()

printf() is a standard library function in C used to display output on the screen.

```c
printf("Hello, World!\n");
```

Function name
printf() is used to print output.

String to be printed
Text inside double quotes is displayed on the screen.

Escape sequence
\n moves the cursor to the next line after printing the text.

Some other examples

Code	Output
`printf("Hello");`	Hello
`printf("C Language");`	C Language
`printf("Number: %d", 5);`	Number: 5
`printf("Value: %.2f", 3.14);`	Value: 3.14
`printf("Hello\nWorld");`	Hello World

3 Compiling and running the program

The C program is first compiled to convert the source code into an executable file. Then the executable file is run to see the output.

1 Write the program

Create and save the file with .c extension (e.g., hello_world.c) in a text editor such as Notepad or VS Code.

2 Compile the program

Windows (using GCC)

```
C:\> gcc hello_world.c -o hello_world.exe
```

Linux / macOS

```
$ gcc hello_world.c -o hello_world
```

This converts the source code into an executable file.

3 Run the program

Windows

```
C:\> hello_world.exe
```

Linux / macOS

```
$ ./hello_world
```

This runs the executable file to get the output.

4 Output

```
Hello_World!
```

The program displays the output on the screen.

4 Common syntax errors

Beginners often make small mistakes. Here are some common errors and how to fix them.

Error	Example	Why it occurs	Fix
Missing semicolon	`printf("Hello")`	Every statement must end with a semicolon (;).	Add ; `printf("Hello");`
Missing header file	`printf("Hello");`	printf() is defined in stdio.h.	Add `#include <stdio.h>`
Misspelled function name	`prinft("Hello");`	Function name is written incorrectly.	Use printf() `printf("Hello");`

Missing braces	`int main()`
	`printf("Hi");`
		Opening { and closing } braces are required.	Add braces
			`int main()`
			`{`
			`printf("Hi");`
			`}`

Unclosed string	`printf("Hello);`	String must be enclosed in double quotes.	Use closing quote `printf("Hello");`

CodeCraft Academy
Empowering Futures with Every Keystroke

3. Structure of a C Program

1 Header files and #include

- Header files contain pre-defined functions, macros and declarations.
- We include a header file using the `#include` directive.
- Example: `stdio.h` provides standard input/output functions like `printf()` and `scanf()`.
- Other commonly used header files:

Header File	Purpose
`stdio.h`	Input/output functions (printf, scanf)
`stdlib.h`	General utilities (e.g., memory allocation, string conversion)
`math.h`	Mathematical functions (sin, cos, sqrt, etc.)
`string.h`	String handling functions (strlen, strcpy, etc.)

2 Structure of a C program

```
/* This program adds two numbers */
#include <stdio.h>
int main()
{
    int a, b, sum;
    printf("Enter two numbers: ");
    scanf("%d %d", &a, &b);
    sum = a + b;
    printf("Sum = %d\n", sum);
    return 0;
}
```

1. **Header file inclusion** — `#include <stdio.h>` Includes standard input/output functions.
2. **main() function** — Execution of the program starts from main().
3. **Variable declaration** — Declare variables to store data.
4. **Statements** — Executable instructions (e.g., input, processing, output).
5. **Return statement** — Indicates successful program completion.

3 Statements, braces and semicolons

- A statement is an instruction in a C program.
- Every statement (except block statements) usually ends with a semicolon (`;`).
- Braces `{ }` are used to group multiple statements into a block (e.g., inside `main()`).
- Indentation improves readability.

Valid statements ✓

```
int x = 10;
float y = 3.5;
printf("Hello");
x = x + 1;
```

Invalid statements ✗

```
int x = 10          ✗
printf("Hello")     ✗
if (x > 0)          ✗
(missing braces)
```

4 Comments and basic syntax

- Comments are used to add notes and explanations in the program. They are ignored by the compiler.
- Single-line comment: `// comment text`
- Multi-line comment: `/* comment text */`
- Comments help in making the program easier to understand.

Example

```
#include <stdio.h>  // header file
int main()
{
    // This is a single-line comment
    printf("Hello, C!");

    /* This is a multi-line comment
       It can span multiple lines */
    return 0;
}
```

5 Compilation and execution

1. Write the program in a text editor



Save with `.c` extension (e.g., program.c)

2. Compile the program using a C compiler

```
C:\> gcc program.c
     -o program
```

This converts the source code into an executable file.

3. Execute the program

```
C:\> program
Enter two numbers: 5 7
Sum = 12
```

Run the executable file to get the output.

4. Output

```
Enter two numbers: 5 7
Sum = 12
```

The program takes input, processes it and displays the result.

4. Algorithm

1 Meaning of an algorithm

An **algorithm** is a finite, ordered set of well-defined steps to solve a problem or perform a task.



Key points:

- It takes some input (zero or more).
- It produces an output (one or more).
- Each step is clearly defined and unambiguous.
- It terminates after a finite number of steps.
- It is independent of any programming language.

Real-life examples:



Steps to make tea/coffee



Steps to reach a destination



Steps to solve a math problem



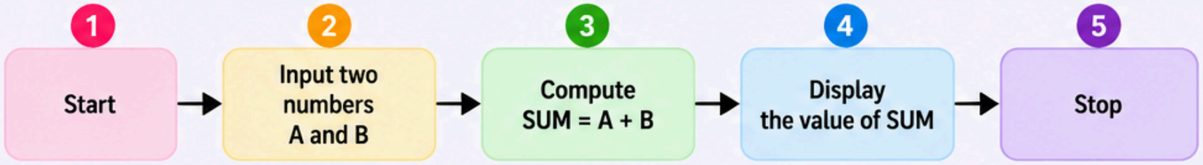
Steps to use a mobile app

2 Characteristics of a good algorithm

A good algorithm should have the following characteristics:

1	**Input**	It should have zero or more inputs clearly specified.
2	**Output**	It should produce at least one output that is clearly specified.
3	**Definiteness**	Every step should be clear, precise and unambiguous.
4	**Finiteness**	It should terminate after a finite number of steps.
5	**Effectiveness**	Each step should be simple and easy to carry out in finite time.
6	**Generality**	It should be applicable to a class of problems, not just a single problem.

Example: Algorithm to find the sum of two numbers



This algorithm has:

- Clearly defined input (two numbers)
- Well-defined steps
- A clear output (sum)
- Finiteness (stops after a few steps)
- Effectiveness (each step is simple)
- Generality (can be used for any two numbers)

5. Flowchart Basics

1 Basic flowchart symbols

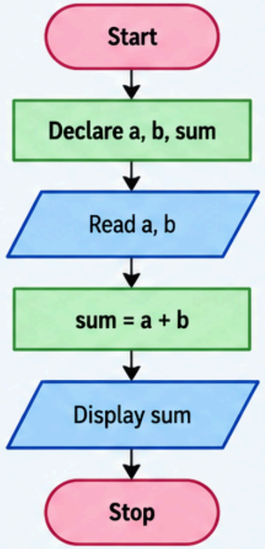
Symbol	Name	Purpose / Meaning
	**Start / Stop**	Indicates the start or end of the flowchart.
	**Input / Output**	Used to read input from the user or display output.
	**Process**	Performs a calculation or processing step.
	**Decision**	Checks a condition and branches to different paths (e.g., Yes/No).
	**Flow line**	Shows the direction of flow.
	**Connector (on-page)**	Used to connect different parts of the flowchart on the same page.
	**Off-page connector**	Used to connect to a part of the flowchart on another page.
	**Predefined process**	Represents a group of predefined steps (e.g., function).

2 Example: Add two numbers

a Algorithm
(Steps to add two numbers)

1. Start
2. Declare variables a, b and sum.
3. Read two numbers a and b from the user.
4. Add the two numbers (sum = a + b).
5. Display the value of sum.
6. Stop

b Flowchart
(Add two numbers)



c C Program
(Equivalent codes)

```c
#include <stdio.h>

int main()
{
   int a, b, sum;
   printf("Enter two numbers: ");
   scanf("%d %d", &a, &b);
   sum = a + b;
   printf("Sum = %d\n", sum);
   return 0;
}
```

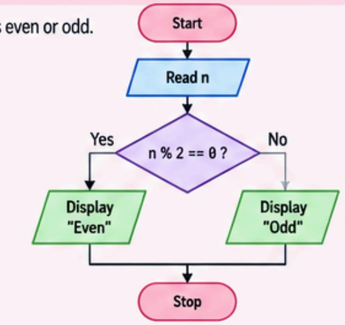
Key Points

- Flowcharts provide a pictorial representation of an algorithm.
- They help in understanding the steps of a problem easily.
- They use standard symbols with fixed meanings.
- Flowcharts are useful before writing the actual program.
- They make the logic clear and help in finding errors.

Another Example

Algorithm: Find whether a number is even or odd.

1. Start
2. Read a number n.
3. Check if n is divisible by 2 (n % 2 == 0).
4. If yes, display "Even", else display "Odd".
5. Stop



Examples of Algorithms and Flowcharts

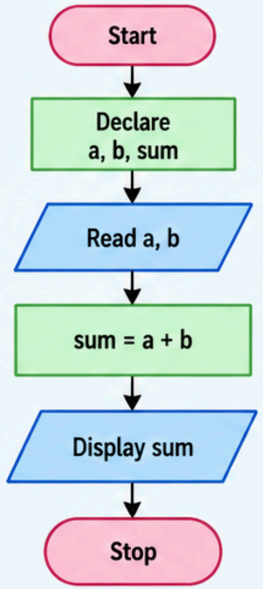
1 Addition of Two Numbers

Problem: Read two numbers and find their sum.

Algorithm

1. Start
2. Declare variables a, b, sum
3. Read two numbers a and b
4. Add the numbers sum = a + b
5. Display the value of sum
6. Stop

Flowchart



Sample Output

```
Enter two numbers: 5 7
Sum = 12
```

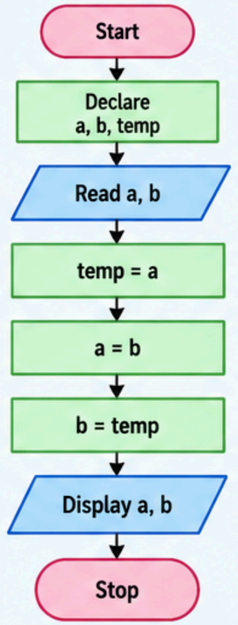
2 Swap of Two Numbers

Problem: Read two numbers and swap their values.

Algorithm

1. Start
2. Declare variables a, b, temp
3. Read two numbers a and b
4. Set temp = a
5. Set a = b
6. Set b = temp
7. Display the new values of a and b
8. Stop

Flowchart



Sample Output

```
Enter two numbers: 10  20
After swapping:
a = 20
b = 10
```

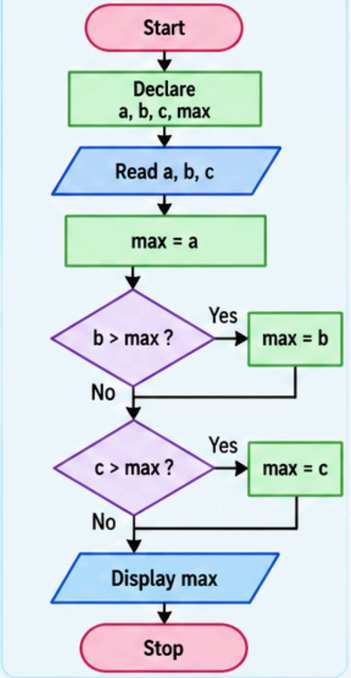
3 Finding the Greatest of Three Numbers

Problem: Read three numbers and find the greatest number.

Algorithm

1. Start
2. Declare variables a, b, c, max
3. Read three numbers a, b and c
4. Set max = a
5. If b > max then set max = b
6. If c > max then set max = c
7. Display the value of max (greatest number)
8. Stop

Flowchart



Sample Output

```
Enter three numbers: 8  15  10
Greatest number = 15
```